

Mysterious Cipher

1. Determine the key

1.1 Data section

- Ciphertext
 - Type: .asciz
 - Initial value: “ddbW”
 - Final value: “ggez” (explanation below)
- num1
 - Type: .long
 - Value: 2
- num2
 - Type: .long
 - Value: -5

1.2 Key Computation

Code	Meaning
<code>movl num1(%rip), %eax</code>	<code>%eax</code> = value stored in num1= 2
<code>movl num2(%rip), %ecx</code>	<code>%ecx</code> = value stored in num2= -5
<code>addl %ecx, %eax</code>	<code>%eax</code> = <code>%eax+%ecx</code> =2+(-5)= -3
<code>movb %al, %bl</code>	<code>%bl</code> = lower 8 bits of <code>%eax</code> = -3

Therefore, the decryption key is -3.

2. Recover the Plaintext

- From characters to number with ASCII
“b=98, d=100, and w=119”
- From the above table %bl= -3
- Start with
 - %rdi = address of ciphertext
 - %rax (%al) =Memory[%rdi]=100
(the 1st character is ‘d’ and in ASCII table=100)
- Note
 - %rdi++ every iteration from decrypt_loop means %rdi moves to the next character if the null terminator has not been reached.

Round	Point at	Operation	character written	Ciphertext After
Initial	–	–	–	ddb\0
1	d	%al=100!=0 (ZF=0) %al=100-(-3)=103=’g’	g	gdb\0 [movb %al, (%rdi)]
2	d	%al=100!=0 (ZF=0) %al=100-(-3)=103=’g’	g	ggbw\0
3	b	%al=98!=0 (ZF=0) %al=98-(-3)=101=’e’	e	ggew\0
4	w	%al=119!=0 (ZF=0) %al=119-(-3)=122=’z’	z	ggez\0
5	0	%al=0 (ZF=1) (Done)	–	ggez

Now Ciphertext’s value becomes “ggez”.

3. Learning

- Assembly organization
 - The “.section .data” is for initializing variables that are used by the program.
 - The “.section .text” is for the executable code that the CPU will run, starting from the “main” function,
- Even though %al is not assigned directly from %rdi, it still equals the value loaded into the lower 8 bits of %rax because %al is part of %rax. Therefore, after movzbq (%rdi), %rax loads one byte from the memory address pointed to by %rdi, %al automatically contains that same byte value.
- Meaning of “.asciz”

- It defines a null-terminated ASCII string. The assembler automatically appends a null character “\0” at the end of the string, allowing functions such as “puts” to determine where the string ends.

4. How I solved the problem

- Firstly, I separated the code into two sections (.data and .text) to see value of each given variable. Then, I traced the key-computation, which I found that the result was copied into “%bl”.
- Secondly, I wrote the rough structure of the loop to see how it works.
- After that, I used the ASCII table to convert the numerical value into characters.
- Finally, I traced the code until it reached the end of the loop.